Klassen und Objekte (OOP I)

BEGRIFFE

Begriff	Bedeutung
Klasse	Bauplan, `class Konto:` (PascalCase)
Objekt / Instanz	konkretes Exemplar, `Konto("A", 100)`
Attribut	Daten, Zustand: `a.stand`
Methode	Funktion in der Klasse: `a.einzahlen(50)`
`__init__`	Konstruktor, legt Attribute an
`self`	das Objekt links vom Punkt

Merksatz: Jedes Objekt hat eigene Attributwerte, die Methoden stehen nur einmal in der Klasse.

KLASSE MIT METHODEN

```
class Konto:
    def __init__(self, inhaber, stand=0):
        self.inhaber = inhaber
        self.stand = stand

    def einzahlen(self, betrag):
        self.stand += betrag

a = Konto("A", 100)
b = Konto("B")
a.einzahlen(50)
print(a.stand, b.stand)  # 150 0
```

- Erzeugen ohne `new`. `a.einzahlen(50)` = `Konto.einzahlen(a, 50)`.
- Genau ein `__init__`, Varianten über Standardwerte.

AUSGABE: STR UND REPR

Methode	für	benutzt von
`__str__`	Menschen	`print()`, f-Strings
`__repr__`	Entwickler	Konsole, Listen

`__str__` gibt mit `return` einen Text zurück. Ohne `__str__` zeigt `print(a)` nur `<__main__.Konto object at 0x…>`.

NAMEN UND KLASSENATTRIBUTE

`k2 = k1` erzeugt kein neues Konto, nur einen zweiten Namen: Änderung über `k2` ist bei `k1` sichtbar, `k1 is k2` ist `True`. Bei eigenen Klassen vergleicht `==` zunächst nur die Identität.

```
class Konto:
    anzahl = 0          # Klassenattribut

    def __init__(self):
        Konto.anzahl += 1  # Klassenname

Konto()
Konto()
print(Konto.anzahl)  # 2
```

KAPSELUNG MIT PROPERTIES

- `_name`: intern (nur Abmachung), `__name`: Name Mangling. Kein echtes `private`.
- Property: Methode, die wie ein Attribut benutzt wird (ohne Klammern).

```
class Konto:
    def __init__(self, stand):
        self._stand = 0
        self.stand = stand  # nutzt Setter

    @property
    def stand(self):         # Getter
        return self._stand

    @stand.setter
    def stand(self, wert):   # Setter
        if wert < 0:
            print("Ungültig")
            return
        self._stand = wert

k = Konto(100)
k.stand = -5      # Ungültig
print(k.stand)    # 100
```

Situation	Lösung
frei änderbar	öffentliches Attribut
nur lesbar	`_attr` + Property ohne Setter
Änderung mit Regeln	`_attr` + Property mit Setter
berechnet	Property ohne eigenes Attribut

HAT-EIN, MODULE, @DATACLASS

- **Hat ein**: Attribut mit anderem Objekt (`self.kunde = kunde`), Verweis, keine Kopie.
- Modul = `.py`-Datei, Paket = Ordner mit `__init__.py`: `from banking.konto import Konto`.

```
from dataclasses import dataclass

@dataclass
class Kunde:
    name: str
    ort: str

k = Kunde("A", "Nord")
print(k)  # Kunde(name='A', ort='Nord')
print(k == Kunde("A", "Nord"))  # True
```

`@dataclass` erzeugt `__init__`, `__repr__`, `__eq__`. Für Datenbehälter; für Verhalten mit Regeln normale Klasse mit Properties.

TYPISCHE FEHLER

- `self` in der Parameterliste vergessen: `TypeError`.
- `self.` vor dem Attribut vergessen.
- In `__str__` `print` statt `return`.
- `self.anzahl += 1` legt ein Instanzattribut an, Klassenzähler bleibt.
- Tippfehler `a.stnad = 500`: still neues Attribut.
- Property mit Klammern aufgerufen: `TypeError`.
- Im Setter `self.stand = …` statt `self._stand`: `RecursionError`.
- `@dataclass` prüft keine Werte.